

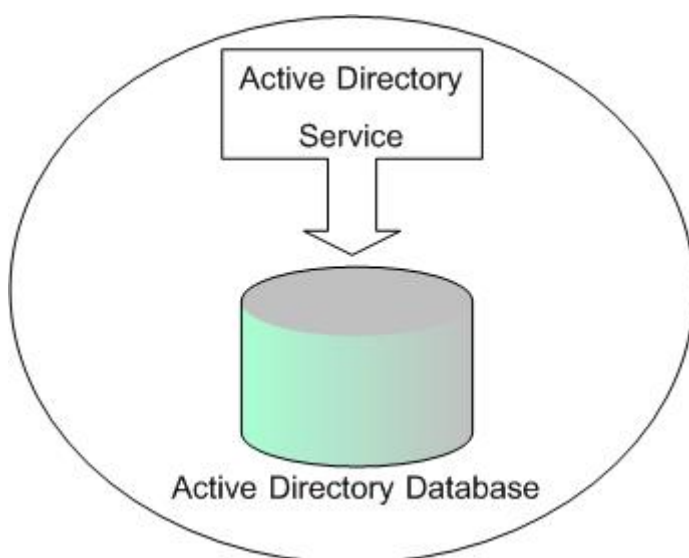
บทที่ 5

Windows 2000 Active Directory

เป็นบริการหนึ่งของ Windows 2000 Server เกี่ยวกับการจัดเก็บข้อมูลของทรัพยากรที่มีอยู่ในเครือข่าย ในไดเรกทอรี โดยทรัพยากรมีความหมายครอบคลุม รายชื่อผู้ใช้ รายชื่อเครื่องที่อยู่ในเครือข่าย รายชื่อเซิร์ฟเวอร์บนไฟล์เซิร์ฟเวอร์ และรายชื่อเครื่องพิมพ์ที่ให้บริการ โดยแอ็กทีฟไดเรกทอรี จะมีฐานข้อมูลในตัวเองสำหรับจัดเก็บรายละเอียดของทรัพยากร

5.1 ส่วนประกอบของวินโดวส์ 2000 แอ็กทีฟไดเรกทอรี

การทำงานแบ่งออกเป็นสองส่วนคือดังรูปที่ 5-1



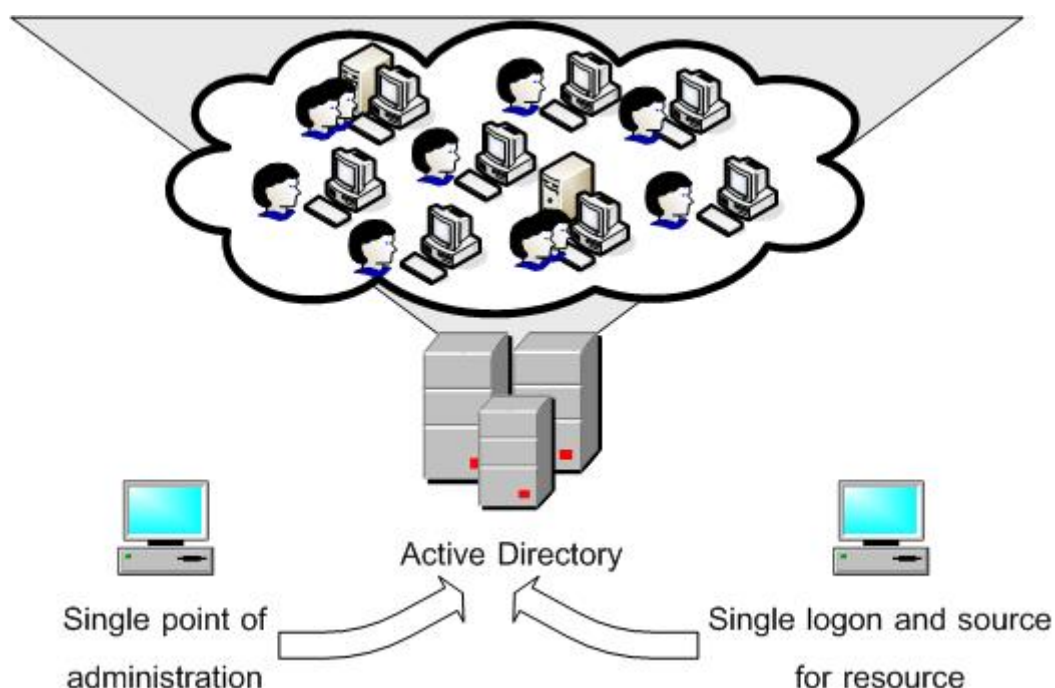
รูปที่ 5-1 แสดงส่วนการทำงานของแอ็กทีฟไดเรกทอรี (Active Directory)

5.1.1 แอ็กทีฟไดเรกทอรีเซอร์วิส (Active Directory Service)

- ให้บริการแก่ผู้บริหารระบบเครือข่ายในการเรียกค้นและจัดการกับบัญชีรายชื่อผู้ใช้ (User Account) และรายชื่อกลุ่ม (Group) ที่จัดเก็บไว้ในแอ็กทีฟไดเรกทอรีดาต้าเบส เช่น การสร้างบัญชีรายชื่อผู้ใช้ใหม่ให้กับผู้ใช้ในองค์กร การตั้งรหัสผ่าน และอื่นๆ รวมถึงให้บริการในการเซตค่า คอนฟิกูเรชัน(configuration) ต่างๆ ให้กับผู้ใช้ ไม่ว่าจะเป็น Domain Policy หรือ Group Policy ต่างๆ ดังนั้น เราจึงสามารถกล่าวได้ว่าแอ็กทีฟเป็นจุดศูนย์กลางในการบริหารระบบเน็ตเวิร์กของ Windows 2000

- อำนาจความสะดวกในการค้นหารายชื่อทรัพยากร ในระบบเครือข่ายที่มีคุณสมบัติ (Attributes) ตามที่ผู้ใช้หรือผู้บริหารระบบต้องการ เช่น ค้นหาว่าเครื่องพิมพ์ใดบ้างที่มีคุณสมบัติ "สามารถพิมพ์สีได้" เป็นต้น แอ็กทีฟไดเรกทอรีเซอร์วิสจะเข้าไปค้นหาจากแอ็กทีฟไดเรกทอรีดาต้าเบส แล้วแสดง

รายชื่อเครื่องพิมพ์ทั้งหมดที่มีคุณสมบัติดังกล่าว มาให้ผู้ใช้เลือก พร้อมทั้งบอกรายละเอียดอื่นๆ ให้ทราบด้วย เช่น เครื่องพิมพ์นี้ติดตั้งไว้ที่เซิร์ฟเวอร์ชื่ออะไร



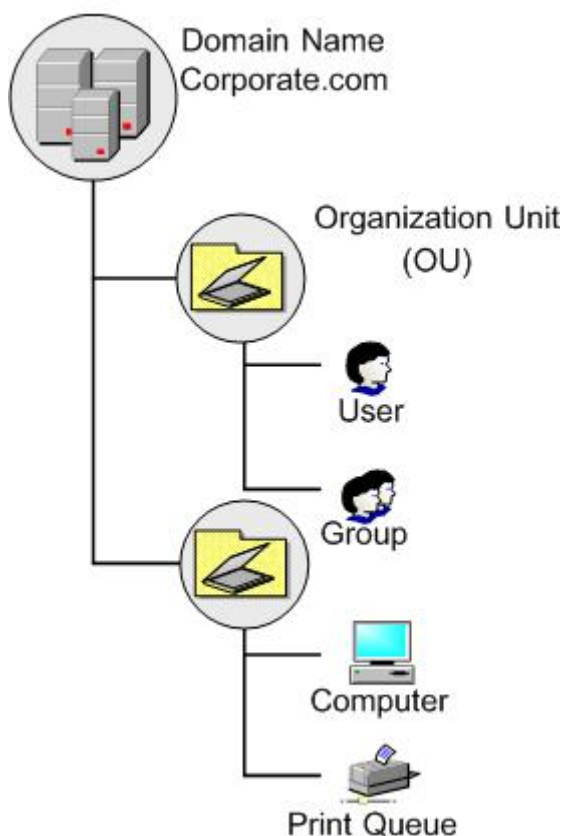
รูปที่ 5-2 การให้บริการของแอคทีฟไดเรกทอรีเซอร์วิส (Active Directory Service)

5.1.2 แอคทีฟไดเรกทอรีดาต้าเบส (Active Directory Database)

ทำหน้าที่ในการจัดเก็บข้อมูลในระบบไดเรกทอรี โดยการเข้าถึงดาต้าเบสจะต้องผ่านการ ล็อกอิน (Login) ถึงจะมีสิทธิ

5.2 ข้อมูลที่ถูกจัดเก็บอยู่ในแอคทีฟไดเรกทอรีดาต้าเบส

- บัญชีรายชื่อผู้ใช้และบัญชีรายชื่อกลุ่ม (User account / Group Account) ของผู้ใช้ในระบบเครือข่ายและรายชื่อเครื่องคอมพิวเตอร์ในเครือข่าย บัญชีรายชื่อเหล่านี้ถือว่าเป็นออบเจกต์ประเภทหนึ่งเหมือนกัน



รูปที่ 5-3 โครงสร้างโดยรวมของแอคทีฟไดเรกทอรีดาต้าเบส (Active Directory Database)

- ออบเจกต์ต่างๆที่ใช้เป็นตัวแทนของทรัพยากรในระบบเครือข่าย เช่น ออบเจกต์ที่เป็นตัวแทนของแชร์โฟลเดอร์(Shared Folder Object) ออบเจกต์ที่เป็นตัวแทนของเครื่องพิมพ์
- คอนเทนเนอร์ต่างๆที่เป็นของระบบตั้งแต่เริ่มต้นเช่นคอนเทนเนอร์ที่ชื่อUsers, Computers และ Built-in
- คอนเทนเนอร์พิเศษที่เรียกว่า Organizational Unit (OU)
- System Configuration ต่างๆ ของโครงสร้าง Active Directory ทั้งหมด เช่น Schema , Global Catalog
- เป็นที่เก็บ Group Policy Object (GPO) เพื่อใช้ในการควบคุม และจัดการเครื่องคอมพิวเตอร์ ของผู้ใช้
- อื่นๆอีกมากมายแล้วแต่จะขยายเพิ่มเติมในอนาคตเช่นออบเจกต์ที่เป็นตัวแทนของ อุปกรณ์สื่อสารในระบบเน็ตเวิร์ก

5.2.1 คอนเทนเนอร์ (Container) คลาส (Class) และ แอตทริบิวต์ (Attributes)

5.2.1.1 คอนเทนเนอร์ (Container)

ผู้บรรจ้ออบเจ็กต์อื่นเพื่อแยกหมวดหมู่ในการออกแบบ ในโครงสร้างของแอ็คทีฟไคเรกทอรีดาต้าเบส ออบเจ็กต์ รายชื่อทรัพยากรทุกอย่างที่อยู่แอ็คทีฟไคเรกทอรีดาต้าเบสจะถูกลองเสมือนว่าเป็น ออบเจ็กต์หนึ่ง ดังนั้น แต่ละออบเจ็กต์ที่เก็บอยู่ในแอ็คทีฟไคเรกทอรีดาต้าเบสจึงถือได้ว่าเป็นตัวแทนของทรัพยากรที่มีอยู่ในระบบเครือข่ายและเป็นที่จัดเก็บรายชื่อผู้ใช้ รายชื่อกลุ่ม รายชื่อเครื่องคอมพิวเตอร์ของผู้ใช้และเครื่องคอมพิวเตอร์ในเน็ตเวิร์ก

5.2.1.2 คลาส (Class)

ออบเจ็กต์จะเป็นตัวแทนประเภทของออบเจ็กต์ ตัวอย่างเช่น รายชื่อผู้ใช้ ทุกรายชื่อที่สร้างขึ้นมาให้กับผู้ใช้ในระบบเครือข่ายจะถูกลองว่าเป็นออบเจ็กต์ที่มีอยู่ในคลาส Users และรายชื่อเครื่องคอมพิวเตอร์ (Computer Account) ในเน็ตเวิร์กก็ถูกลองว่าเป็นออบเจ็กต์ที่จัดอยู่ในคลาส Computer นอกจากนั้น ทุกออบเจ็กต์ที่อยู่ในคลาสเดียวกันจะต้องมีแอตทริบิวต์ และข้อมูลกำหนด (definition) ต่างๆ เหมือนกัน

5.2.1.3 แอตทริบิวต์ (Attribute)

ใช้ในการบ่งบอกคุณลักษณะ คุณสมบัติ และพฤติกรรมต่างๆ ของออบเจ็กต์ โดยออบเจ็กต์ทุกตัวที่อยู่ในคลาสเดียวกันจะต้องมีแอตทริบิวต์ทุกอย่างเหมือนกัน เช่น ออบเจ็กต์ในคลาส User (รายชื่อผู้ใช้) จะมีแอตทริบิวต์ First Name ,Last Name, Logon Name (ชื่อที่ใช้ในการล็อกออนเข้าสู่เน็ตเวิร์ก), Password (รหัสผ่าน) และอื่นๆ เหมือนกัน แต่สิ่งที่ทำให้ออบเจ็กต์แต่ละตัวแตกต่างกันไปก็คือ ค่า (value) ของแอตทริบิวต์

5.2.2 สกีมา (Schema)

เป็นฐานข้อมูลส่วนย่อยที่เก็บอยู่ในแอ็คทีฟไคเรกทอรีดาต้าเบส ซึ่งทำหน้าที่เก็บรวบรวมข้อกำหนดเกี่ยวกับคลาสของออบเจ็กต์ทุกคลาส เช่น ออบเจ็กต์ที่อยู่ในแต่ละคลาสจะต้องมีแอตทริบิวต์อะไร ค่าของแต่ละแอตทริบิวต์มีคุณสมบัติเป็นอะไร Schema ในแอ็คทีฟไคเรกทอรีดาต้าเบสของวินโดวส์ 2000 สามารถเพิ่มขยายได้โดยผู้บริหารระบบเครือข่าย ได้แก่ การเพิ่มคลาสประเภทใหม่ๆ เข้าไป หรือการเพิ่ม แอตทริบิวต์ใหม่ให้กับคลาสเดิมที่มีอยู่แล้ว

5.2.3 ออกาไนเซชันยูนิต (Organization Unit)

จะทำหน้าที่เสมือนคอนเทนเนอร์ ถูกใช้เพื่อจัดแบ่งออบเจ็กต์ต่างๆ ให้เป็นกลุ่ม สอดคล้องกับโครงสร้างตามลำดับชั้นของการบริหารหน่วยงานหรือแผนกต่างๆ ในองค์กร หรือให้สอดคล้องกับที่ตั้งของหน่วยงาน/สาขาต่างๆ ภายในองค์กร

ประโยชน์ที่ได้รับเมื่อมีการสร้าง OU ขึ้นมาเพื่อจัดแบ่งกลุ่มของออบเจ็กต์

- ผู้บริหารระบบโดเมนสามารถจัดระเบียบและมองเห็นรายชื่อผู้ใช้ได้ง่ายว่า

- ลดภาระในการจัดการทรัพยากร กล่าวคือ สามารถจัดการกับออบเจกต์เป็นกลุ่มได้ทีเดียว เช่น การกำหนดนโยบายการรักษาความปลอดภัย (Security Policy) และนโยบายในการติดตั้งซอฟต์แวร์ (Software Group Policy) ให้กับออบเจกต์ประเภท User หรือ Computer ที่อยู่ภายใต้ OU นั้นๆ

- ผู้บริหารระบบสามารถกำหนดสิทธิ (Permission) ต่างๆ ของผู้ใช้ในการเข้าถึงออบเจกต์ผ่านทาง OU ได้

- ผู้ดูแลโดเมนในระดับบนสุดสามารถกระจายอำนาจการจัดการให้ผู้ดูแลแผนกหรือ ภูมิภาคนั้นๆ เป็นผู้มีความรับผิดชอบในการจัดการกับรายชื่อผู้ใช้ และทรัพยากรต่างๆ ภายใต้แผนกของตนได้อย่างเต็มที่ ลักษณะนี้เรียกว่า "Delegate Administrative Control" ซึ่งทำได้โดยการกำหนดให้ผู้รับผิดชอบแผนกนั้นมีสิทธิสมบูรณ์บน OU ที่ตนต้องรับผิดชอบ จากนั้นเขาสามารถจัดการกับออบเจกต์ที่อยู่ภายใต้ OU นั้น อย่างเต็มที่

5.2.4 โดเมน (Domain)

โดเมนเป็นการรวมกลุ่มทรัพยากรต่างๆ ในระบบเครือข่ายต่างๆ ตั้งแต่ เครื่องพิมพ์ แฟกซ์โพลเดอร์ เครื่องคอมพิวเตอร์ เครื่องเซิร์ฟเวอร์ รายชื่อผู้ใช้เข้าไว้ด้วยกันให้มีความหมายในเชิงบริหาร และการจัดการ โดยทรัพยากรต่างๆ ข้างต้นจะต้องอยู่ภายใต้ชื่อโดเมนเดียวกัน และชื่อโดเมนจะบ่งบอกให้เห็นถึงชื่อของหน่วยงานหรือชื่อองค์กรนั้นๆ ซึ่งผู้บริหารระบบโดเมนจะมีความสามารถเต็มที่ในการจัดทรัพยากรต่างๆ ที่อยู่ภายใต้โดเมน

โดเมนยังถูกมองได้ว่าเป็นเสมือนขอบเขตของระบบรักษาความปลอดภัย (Security Boundary) และขอบเขตของการบริหาร (Administrative Boundary) ด้วย ดังจะเห็นได้จากข้อเท็จจริงต่อไปนี้

- ผู้ใช้ เมื่อผู้ใช้เข้ามาในโดเมนใดโดเมนหนึ่งเรียบร้อยแล้ว ผู้ใช้สามารถใช้ทรัพยากรต่างๆ ทั้งหมดที่อยู่ภายใต้โดเมนนั้นได้ตามสิทธิที่ตัวเองมี แต่ผู้ใช้อาจไม่สามารถเข้าไปใช้งานทรัพยากรที่ติดตั้งอยู่ภายใต้โดเมนอื่นได้ จนกว่าผู้บริหารระบบของโดเมนอื่นจะกำหนดสิทธิอนุญาตไว้ให้

- ผู้บริหารระบบ มีสิทธิเต็มที่ในการจัดการบัญชีรายชื่อผู้ใช้และทรัพยากรต่างๆ ได้เฉพาะภายในโดเมนของตนเองเท่านั้น ไม่มีสิทธิเข้าไปดูแลหรือจัดการกับโดเมนอื่น (ยกเว้นว่าผู้บริหารโดเมนอื่นจะได้อนุญาตเป็นพิเศษ)

ภายในโดเมนหนึ่งๆจะมีฐานข้อมูลส่วนกลางที่ทำหน้าที่จัดเก็บรวบรวมรายละเอียดของทรัพยากรและรายชื่อต่างๆ ข้างต้น เพื่อให้เครื่องที่ทำหน้าที่เป็นเซิร์ฟเวอร์ทุกตัวในระบบสามารถอ้างอิง และใช้งานฐานข้อมูลส่วนกลางนี้ได้ ซึ่งถูกจัดเก็บไว้ที่เครื่อง Windows 2000 Server ที่ทำหน้าที่เป็น Domain Controller (โดเมนคอนโทรลเลอร์)

โหมดของโดเมน (Domain Modes) แบ่งออกได้เป็น 2 โหมดคือ

1. Mixed Mode หลังจากติดตั้งแอคทีฟไดเรกทอรีเพื่อสร้างโดเมนของ Windows 2000 ขึ้นมาแล้ว โหมดของโดเมนจะถูกกำหนดให้เป็น Mixed Mode เพื่อให้โดเมนคอนโทรลเลอร์ของ Windows 2000 สามารถซิงโครไนซ์บัญชีรายชื่อผู้ใช้และสื่อสารกับเครื่อง Backup Domain Controller (BDC) ของ Windows NT 4.0 ที่ทำงานอยู่ในโดเมนเดียวกันได้ จุดประสงค์อันนี้ก็เพื่อรองรับการอัปเกรดจากเน็ตเวิร์ก

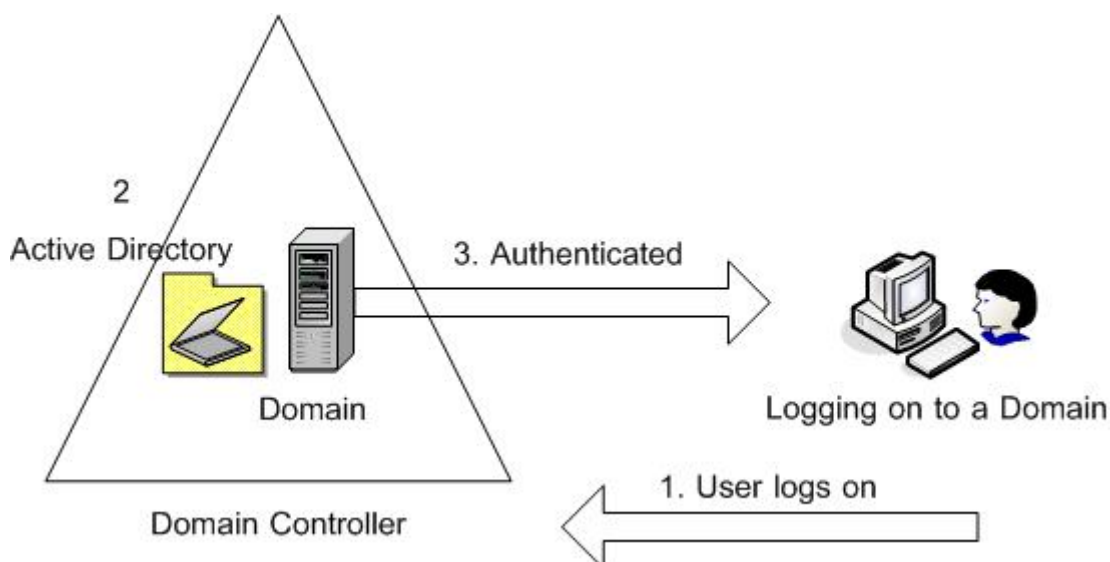
ของ NT 4.0 มาเป็น Windows 2000 นั่นเอง เพราะในการอัปเกรด เราจะเริ่มอัปเกรดที่เครื่อง PDC ของ Windows NT 4.0 มาเป็นโดเมนคอนโทรลเลอร์ของ Windows 2000 ก่อน โดยที่เครื่อง BDC ยังคงทำงานอยู่บน Windows NT Server 4.0

2. Native Mode หลังจากที่เราที่เครื่องโดเมนคอนโทรลเลอร์ทุกตัวได้รับการอัปเกรดจาก NT 4.0 มาเป็นโดเมนคอนโทรลเลอร์ที่ทำงานบน Windows 2000 Server แล้ว เราสามารถเปลี่ยนจาก Mixed Mode ให้เป็น Native Mode ได้ ซึ่งการทำงานใน Native Mode มีข้อได้เปรียบกว่า Mixed Mode หลายเรื่องของการจัดรายชื่อกลุ่มซ้อนกลุ่ม หรือที่เรียกว่าการทำ Group Nesting และการประยุกต์ใช้รายชื่อกลุ่มพิเศษที่ชื่อ Universal Group Nesting

5.2.5 โดเมนคอนโทรลเลอร์ (Domain Controller)

โดเมนคอนโทรลเลอร์จะมีหน้าที่ตรวจสอบรายชื่อผู้ใช้ที่ต้องการใช้งานระบบตรงกับบัญชีรายชื่อผู้ใช้ในแอ็กทีฟไดเรกทอรีดาต้าเบสหรือไม่ ก่อนจะอนุญาตให้ ผู้ใช้เข้ามาใช้งานทรัพยากรต่างๆ ในระบบได้ และเมื่อมีผู้ใช้งานระบบเสร็จเรียบร้อยแล้ว ผู้ใช้สามารถติดต่อเข้าไปใช้ทรัพยากร (เซิร์ฟเวอร์ เครื่องพิมพ์) จากเซิร์ฟเวอร์ใดก็ได้ที่อยู่ภายใต้โดเมนนั้นๆ ส่งผลให้ผู้ใช้สามารถใช้รายชื่อเดียวเพื่อการเข้าถึงทรัพยากรและเซิร์ฟเวอร์ต่างๆ ได้ทั้งหมด ลักษณะนี้เรียกว่า "Single Network Logon" อีกทั้งผู้บริหารระบบเครือข่ายก็ได้รับความสะดวกด้วย เพราะเสียเวลาในการสร้างรายชื่อผู้ใช้ของผู้ใช้ในโดเมนขึ้นมาเพียงครั้งเดียวแล้วจัดเก็บในแอ็กทีฟไดเรกทอรีดาต้าเบส จากนั้นไม่ว่าในอนาคตจะมีเซิร์ฟเวอร์เกิดขึ้นมากี่ตัว ผู้บริหารเครือข่ายก็ไม่จำเป็นต้องไปเสียเวลาสร้างรายชื่อผู้ใช้ขึ้นมาใหม่ที่เซิร์ฟเวอร์เหล่านั้นอีก เพียงแค่ติดตั้งให้เซิร์ฟเวอร์เหล่านั้นเข้ามาเป็นสมาชิกของโดเมนที่มีอยู่แล้วเท่านั้นเอง เซิร์ฟเวอร์เหล่านี้จะรู้จักรายชื่อผู้ใช้ทั้งหมดในโดเมนทันที

ในรูปที่ 5-4 เมื่อผู้ใช้ขอเข้าใช้งานระบบ (ขั้นที่ 1) โดเมนคอนโทรลเลอร์จะตรวจสอบชื่อผู้ใช้ที่ระบบเข้ามาอยู่ในแอ็กทีฟไดเรกทอรีดาต้าเบสหรือไม่ และรหัสผ่านที่ระบุถูกต้องหรือไม่ (ขั้นที่ 2) ถ้าถูกต้อง โดเมนคอนโทรลเลอร์ยอมรับผู้ใช้ และอนุญาตให้ใช้ทรัพยากรต่างๆ ภายใต้โดเมนได้ ในขั้นนี้ถือว่าผู้ใช้ผ่านตรวจสอบจากโดเมนคอนโทรลเลอร์แล้ว (ขั้นที่ 3) ผ่านการพิสูจน์ตน (Authenticate) โดเมนคอนโทรลเลอร์มีหน้าที่ควบคุมการให้บริการแอ็กทีฟไดเรกทอรี เก็บรักษาและดูแลฐานข้อมูลสำคัญของโดเมนในแอ็กทีฟไดเรกทอรีดาต้าเบส



รูปที่ 5-4 แสดงการทำงานของการทำงานระบบบนโดเมน

5.2.6 Replication (เรพลิเคชัน)

แอ็กทีฟไดเรกทอรีคือฐานข้อมูลที่จัดเก็บอยู่บนเครื่องโดเมนคอนโทรลเลอร์เป็นฐานข้อมูลที่มีความสำคัญมากต่อการทำงานของระบบโดเมน ไม่ควรซอฟต์แวร์จึงได้ออกแบบให้ใน 1 โดเมนสามารถมีเครื่องโดเมนคอนโทรลเลอร์ได้มากกว่า 1 ตัว โดยที่ทุกๆตัวเก็บแอ็กทีฟไดเรกทอรีข้อมูลเดียวกัน ในยามปกติทุกๆ โดเมนคอนโทรลเลอร์จะช่วยกันทำหน้าที่ตรวจสอบการเข้าใช้งานระบบของผู้ใช้ และให้บริการ แอ็กทีฟไดเรกทอรีดังที่ได้กล่าวไปข้างต้น แต่ในกรณีที่ตัวใดตัวหนึ่งเกิดปัญหา ตัวอื่นที่เหลืออยู่ก็สามารถให้บริการแก่ผู้ใช้ในโดเมนต่อไปได้ เพื่อให้แอ็กทีฟไดเรกทอรีคือฐานข้อมูลที่เก็บไว้บนแต่ละโดเมนคอนโทรลเลอร์ในโดเมนมีความสอดคล้องกันอยู่ตลอดเวลา โดเมนคอนโทรลเลอร์จะมีระยะอัปเดตข้อมูลในแอ็กทีฟไดเรกทอรีคือฐานข้อมูล เรียกว่า การเรพลิเคชัน ระหว่างโดเมนคอนโทรลเลอร์

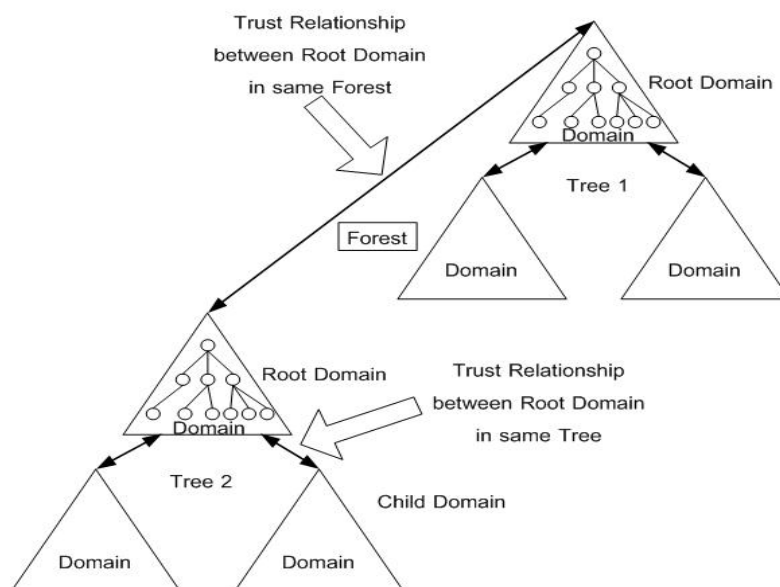
5.2.7 Trust Relationship

Trust Relationship เป็นเสมือน "Communication Link" เชื่อมโยงระหว่าง 2 โดเมน โดยการที่โดเมนหนึ่งเชื่อถืออนุญาตให้ผู้ใช้ของโดเมนอื่นสามารถเข้ามาใช้งานทรัพยากรที่อยู่ในโดเมนของตนได้ ผ่านเน็ตเวิร์กที่หน้าจอยเซิร์ฟเวอร์ในโดเมนของตน โดยอาศัยการพิสูจน์ตนจากแอ็กทีฟไดเรกทอรีคือฐานข้อมูลบนโดเมนคอนโทรลเลอร์ในโดเมนเดิมของผู้ใช้คนนั้นๆ Trust Relationship จึงเป็นสิ่งที่ใช้ในการแลกเปลี่ยนทรัพยากรระหว่างโดเมน หรือเราสามารถกล่าวได้อีกอย่างหนึ่งว่า Trust Relationship เป็นเครื่องมือที่ใช้สร้างความสัมพันธ์ระหว่างโดเมนกับโดเมน

5.2.8 ทรี (Tree) และ ฟอว์เรสต์ (Forest)

ทรี (tree) เป็นการรวมกลุ่มโดเมนเข้าด้วยกันผ่านทาง Trust Relationship เพื่อให้เกิดการแลกเปลี่ยนการใช้งานทรัพยากรข้ามระหว่างโดเมนในองค์กรเดียวกัน โดเมนที่อยู่ในระดับบนสุดของทรี

เรียกว่า Parent Domain และ โดเมนที่อยู่ข้างใต้ Parent Domain จะถูกเรียกว่า Child Domain ฟอว์เรสต์เป็นการรวมกลุ่มทรีหลายๆ ทรีเข้าด้วยกันดังแสดงในรูป เพื่อการแลกเปลี่ยนทรัพยากรระหว่างทรีโดยทั่วไปการสร้างฟอว์เรสต์จะถูกใช้เพื่อการสื่อสารระหว่างสององค์กรมากกว่า อย่างเช่น บริษัทหนึ่งก็จะนำเอาทรีของตนมาเชื่อมโยง (Join) เป็นฟอว์เรสต์



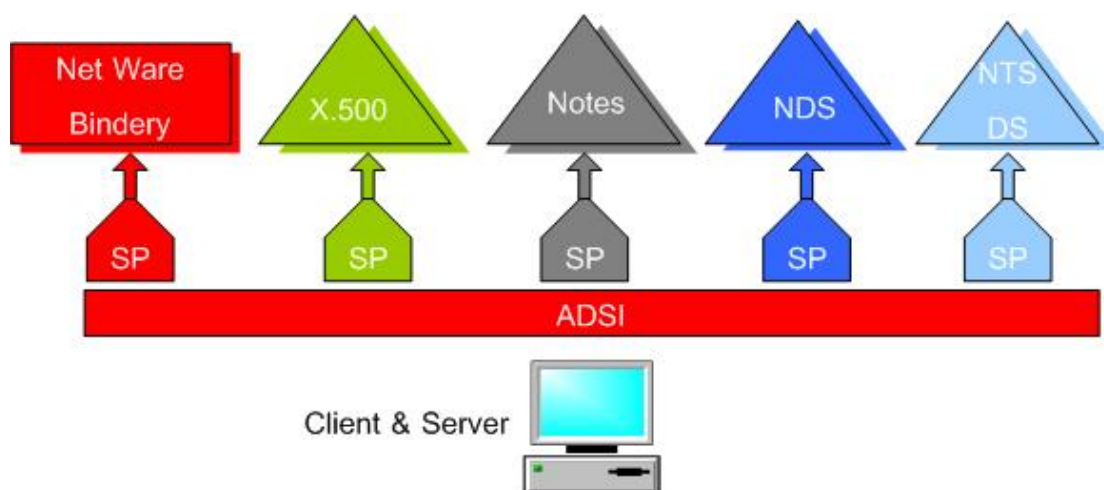
รูปที่ 5-5 แสดงโครงสร้างของ ทรี (Tree) และ ฟอว์เรสต์ (Forest)

5.2.9 ไซต์ (Site)

การจัดรวม IP Subnet เข้าไว้ด้วยกัน โดยที่ Subnet ทั้งหลายเหล่านั้นเชื่อมต่อกันด้วยเน็ตเวิร์กความเร็วสูง การกำหนดไซต์จะช่วยให้ผู้บริหารระบบสามารถกำหนด โทโปโลยี (Topology) ของการเรพลีเคชันระหว่าง เครื่องโดเมนคอนโทรลเลอร์ได้ และกำหนดกลุ่มของเครื่องโดเมนคอนโทรลเลอร์ที่ทำหน้าที่ตรวจสอบการเข้าใช้ระบบของผู้ใช้ที่อยู่ในพื้นที่เดียวกัน

5.3 Active directory Service Interfaces (ADSI)

เป็น API ใช้เข้าถึงข้อมูลในไดเรกทอรีเซอวิซ สำหรับระบบเน็ตเวิร์กที่แตกต่างกัน ดังรูปที่ 5-6 ดังนั้นเราใช้ ADSI อย่างเดียวกันก็สามารถเข้าไปบริหารทรัพยากรในระบบเน็ตเวิร์กที่ต่างระบบการจัดการไม่ว่าเป็น NetWare Bindery, X.500, Notes, NDS และ NTS DS ผู้ดูแล หรือผู้พัฒนา สามารถใช้ ADSI ค้นหาหรือจัดการทรัพยากรในไดเรกทอรีเซอวิซ และง่ายต่อการนำไปใช้งาน



รูปที่ 5-6 การใช้ ADSI เข้าถึงข้อมูล

5.3.1 Active Directory Service Interfaces Architecture

ADSI ประกอบด้วย ADSI object และ Dependent object ผู้ใช้สามารถกระทำงานต่างๆ ผ่านทาง Interfaces ADSI สร้าง ADSI Object และหลาย Interface

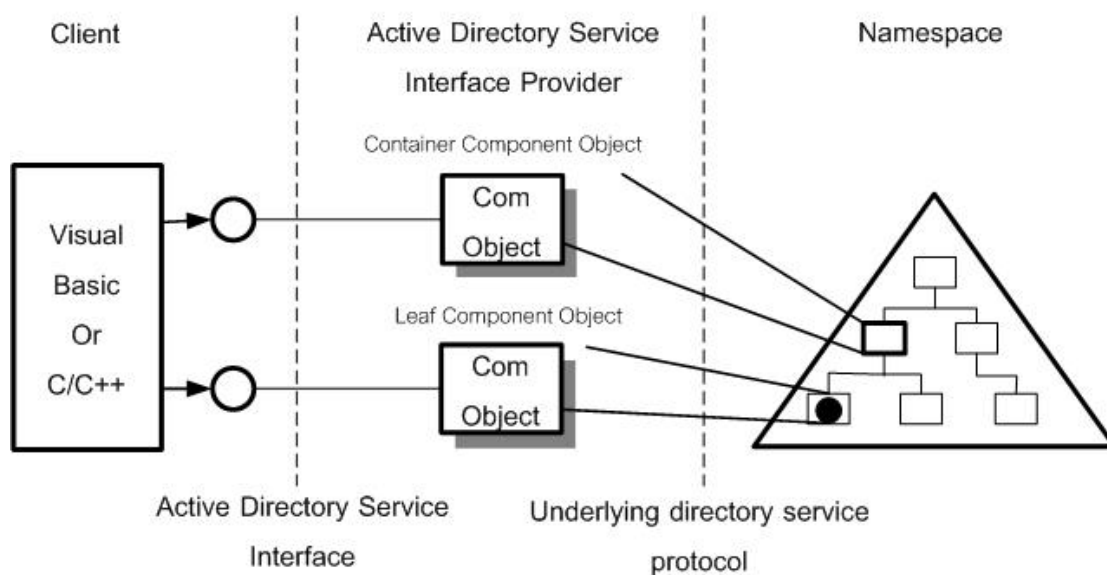
Active Directory Service Interfaces Object

เป็น COM ที่ทำงานอยู่ภายใต้ไคเรกทอรีเซอร์วิสสามารถใช้งาน ADSI ผ่านทาง COM ADSI object แบ่งออกเป็น 2 กลุ่มดังนี้

1. directory service leaf objects สามารถบรรจุ ADSI object อื่นได้
2. directory service container objects ไม่สามารถบรรจุ ADSI object อื่นได้

5.3.2 Active Directory Service Interfaces Provider

ADSI สร้างมาให้บรรจุ ADSI object และ dependent objects ทำหรับ Namespace โดยเฉพาะ จากรูปที่ 5-7 แสดงถึงผู้ใช้เกี่ยวกับการค้นหาหรือใช้ข้อมูลผ่านทาง Interfaces เท่านั้นโดยไม่ต้องรู้รายละเอียดภายใน



รูปที่ 5-7 โครงสร้าง Provider

5.3.3 Active Directory Service Interfaces Schema Management

ADSI สร้างให้สามารถเข้าถึง namespaces ที่แตกต่างกันได้ด้วยการเขียนที่เหมือนกัน อย่างไรก็ตามในการเข้าถึงไดเรกทอรีอาจจะใช้หลายฟังก์ชันขึ้นอยู่กับรายละเอียดเฉพาะของADSI ไดเรกทอรีอาจจะไม่ได้ประกาศวัตถุที่บรรจุเหมือน ADSI อื่นๆ ในการเพิ่ม ต่อเติมข้อมูลในไดเรกทอรีเซอว์วิส อนุญาตให้แก้ไขได้เฉพาะ Schema พื้นฐาน และสามารถเพิ่มเติมได้ตามอำเภอใจของผู้ดูแลและอิสระจาก ผู้ขายเราสามารถขยายวัตถุด้วย Schema Management Active Directory Service Interfaces objects ใช้ได้ ดังนี้

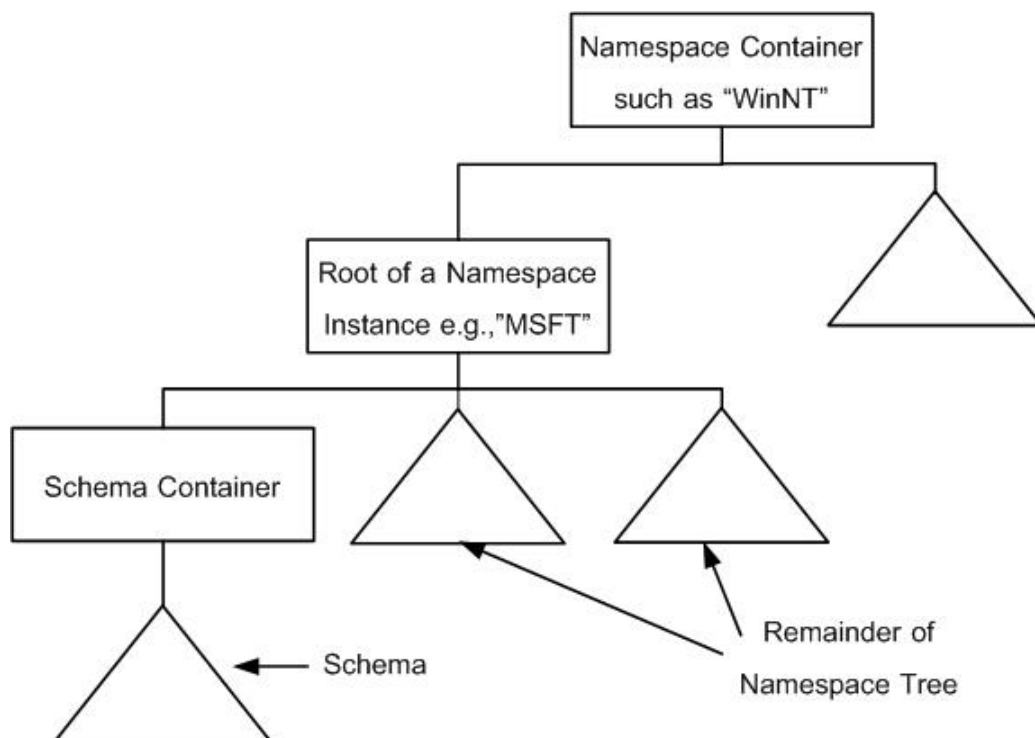
- เรียกดูข้อมูลวัตถุที่ประกาศไว้แล้ว
- ขยายข้อมูลวัตถุที่ประกาศไว้

5.3.4 Schema Management Active Directory Service Interfaces Objects

สามารถใช้ค้นหาและแก้ไขข้อมูลของ Schema ของ Namespace ประกอบด้วยรายละเอียดดังนี้

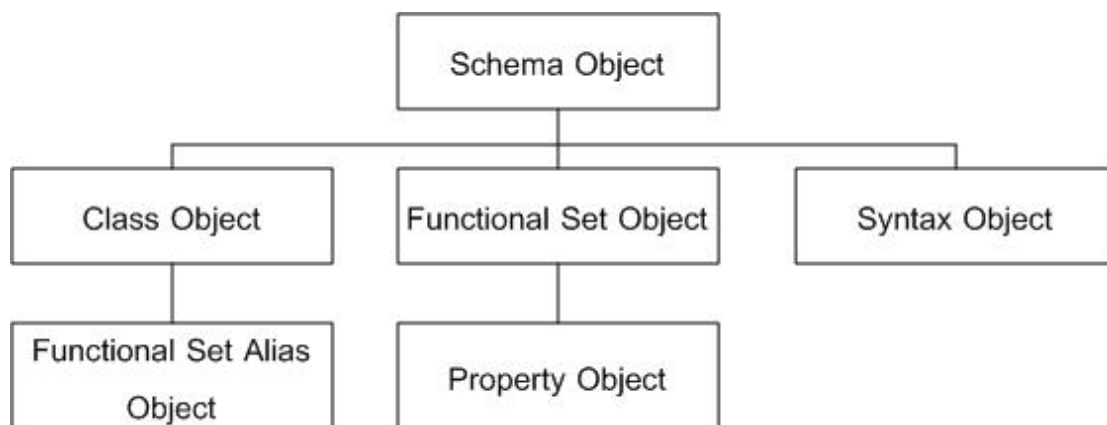
- Schema container object

ใช้รวมวัตถุที่ประกาศไว้ในส่วนหนึ่งของ ไดเรกทอรี โดยทั่วไปทุกวัตถุจะต้องมี schema เป็นของตัวเอง ADSI จะแสดงโดยที่การวาง schema container บนลูกของ directory root



รูปที่ 5-8 Schema containers

รูปที่ 5-8 แสดงถึงที่อยู่โดยทั่วไป อย่างไรก็ตาม ADSI ไม่ได้จำกัดว่าจะอยู่ตรงไหนของไคลเรกทอรี ความซับซ้อนของไคลเรกทอรีอาจจะอนุญาตให้มี schema ออกเป็นหลายส่วนดังรูปที่ 4



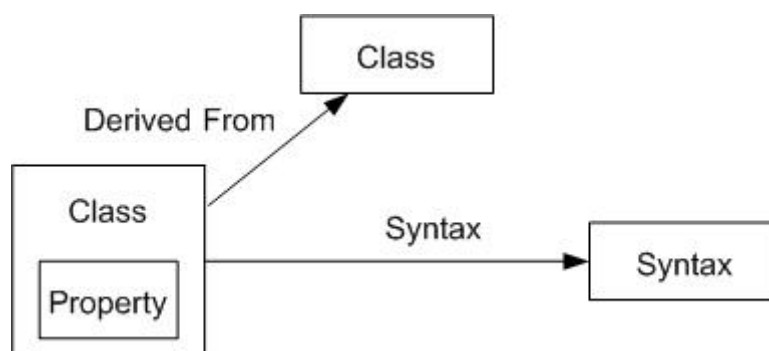
รูปที่ 5-9 Schema hierarchy

schema container ตัวมันเองจะประกอบด้วย class, functional set, property, และ syntax ดังรูปที่

5-9

- Class container object

ใช้ในการประกาศ Class ที่สร้างใน ไดรเรททอรี Classใหม่สามารถถอดคุณสมบัติจากClass อื่นที่ใช้อยู่ในไดรเรททอรี



รูปที่ 5-10 Creating a class

รูปที่ 5-10 แสดงให้เห็นถึงความสัมพันธ์ของ คลาสอื่น คุณสมบัติ และการสร้างเป็นคลาสใหม่

5.3.5 Active Directory Service Interfaces Caching

ทุก วัตถุ ADSI จะมีสอง methods คือ GetInfo และ SetInfo เพื่อให้ง่ายต่อการเข้าถึงข้อมูลคุณสมบัติ การทำการอ่าน หรือกำหนดค่าคุณสมบัติจะทำงานอยู่ใน cache

เราสามารถเรียกใช้คุณสมบัติจาก วัตถุ ADSI ได้ทุกเวลาหลังจากที่มีการเข้าถึงวัตถุแล้ว ในการเรียกเราต้องเรียก GetInfo ก่อน ถ้าคุณสมบัติไม่ได้มีการถามมาก่อนจะอ่านข้อมูลมาให้และเก็บไว้ใน Cached และเมื่อมีการถามอีกจะอ่านจาก Cached

- GetInfo จะเป็นการเรียกให้มีการอ่านข้อมูลที่แน่นอนออกมา
- SetInfo จะเป็นการเขียนข้อมูลลงไดรเรททอรี

5.3.6 Active Directory Service Interfaces Names

ชื่อที่ใช้เรียกวัตถุในไดรเรททอรีจะมีเพียงชื่อเดียว ซึ่งจริงๆแล้วจะมีการจัดลำดับตามไดรเรททอรีเพื่อให้หาได้ง่าย ดังตัวอย่าง

CN=JSmith, OU=Sales, DC=ArcadiaBay, DC=Com

จุดหมายในการลดขนาดของความรู้ในโค้ดที่ต้องมีสำหรับที่อยู่ของวัตถุ ซึ่งเราจะเรียกที่อยู่ของวัตถุว่า ADsPath ทำให้เกิดการเรียกที่อยู่ของวัตถุได้หลายแบบ ดังนี้

WinNT://REDMOND/jsmith

WinNT://REDMOND/comp1, computer

WinNT://REDMOND/comp1/alice

LDAP://OU=Sales, DC=ArcadiaBay, DC=COM

LDAP://exch01/O=Microsoft

5.3.7 Active Directory Service Interfaces Navigation

ADSI ถูกสร้างให้ง่ายโดนใช้ผ่าน IADsContainer การใช้interface สามารถเรียกใช้เฉพาะวัตถุที่สนใจ

ตัวอย่างการเข้าถึง organizational unit

```
Set ou = GetObject("LDAP://host1/OU=Sales, DC=ArcadiayBay,DC=COM")
```

```
For each obj in ou
```

```
    Debug.Print obj.Name
```

```
Next
```

ตัวอย่าง แสดงรายชื่อผู้ใช้ใน ArcadiaBay โดเมน

```
Set dom = GetObject("WinNT://ArcadiaBay")
```

```
dom.Filter = Array("user")
```

```
For each usr in dom
```

```
    Debug.Print usr.Name
```

```
Next
```

5.3.8 Active Directory Service Interfaces Searching

การค้นหาข้อมูลเป็นอย่างหนึ่งที่มีการใช้งานจากผู้ใช้อยู่ในหลายไดเรกทอรีใน ADSI สามารถค้นหาโดยใช้ผ่านทาง OLE DB interfaces หรือ ADSI สำหรับการควบคุมที่ดีในการค้นหาจะต้องมีการกำหนดในส่วนของการละเอียดอื่นเช่น search page size, sort, size limit, search level, search scope และ other options

ตัวอย่าง การอ่านค่าจาก Active Directory โดนใช้ ADO

```
Dim con As New Connection, rs As New Recordset
```

```
Dim Com As New Command
```

```
con.Provider = "ADsDSOObject"
```

```
con.Open "Active Directory Provider"
```

```
Set Com.ActiveConnection = con
```

```
Com.CommandText = "select name from 'LDAP://DC=ArcadiayBay,DC=COM' where
```

```
objectClass='*' ORDER BY NAME"
```

```
Com.Properties("Page Size") = 1000
```

```

Com.Properties("Timeout") = 30 'seconds
Com.Properties("searchscope") = ADS_SCOPE_SUBTREE

Set rs = Com.Execute

While Not rs.EOF
    Debug.Print rs.Fields("Name").Value
    rs.MoveNext
Wend

```

5.3.9 Active Directory Service Interfaces Security

ADSI ถูกสร้างมาให้สามารถใช้ได้ทั้งการพิสูจน์ตนและพิสูจน์สิทธิของผู้ใช้ที่เข้ามาใช้ ถ้าผู้ใช้เข้ามาใช้งาน Active Directory ผู้ใช้อาจให้การพิสูจน์ด้วย Kerberos หรือ NTLM ขึ้นอยู่กับโครงสร้างที่เหมาะสม

ตัวอย่าง JSmith เข้ามาใช้งานระบบ

```

Dim dso As IADsOpenDSObject
Dim domain As IADsDomain
Set dso = GetObject("WinNT:")
Set domain = dso.OpenDSObject("WinNT://ArcadiaBay", "JSmith", "secret",
ADS_SECURE_AUTHENTICATION)

```

ตัวอย่าง การเพิ่ม ACE ด้วยสิทธิของวัตถุ

```

Dim Ace1 as new IADsAccessControlEntry
Dim Ace2 As new IADsAccessControlEntry
Dim Dacl as new IADsAccessControlList
Dacl.AclRevision = 4 'DS ACL Revision
Ace1.AccessMask = -1 'Full Permission (Allowed)
Ace1.AceType = ADS_ACETYPE_ACCESS_ALLOWED
Ace1.AceFlags = ADS_ACEFLAG_INHERIT_ACE
Ace1.Trustee = "ACTIVED\Administrator"
Dacl.AddAce Ace1
Dacl.AddAce Ace2

```